

# Backend Architecture

---

The backend is a .NET 10 ASP.NET Core Web API following Clean Architecture. The four layers never reference each other in the wrong direction: Domain has no dependencies, Application depends only on Domain, Infrastructure implements Application's interfaces, and the API wires everything together.

---

## Project Structure

---

```
FoodLoop.sln
├── src/
│   ├── FoodLoop.Domain           ← Entities, enums, domain
│   ├── FoodLoop.Application      ← Commands, queries, DTOs
│   ├── FoodLoop.Infrastructure   ← EF Core, Identity, JWT
│   └── FoodLoop.API              ← Controllers, middleware
└── test/
    ├── FoodLoop.Domain.Tests
    ├── FoodLoop.Application.Tests
    └── FoodLoop.Infrastructure.Tests
```

## Layer Responsibilities

---

### Domain (**FoodLoop.Domain**)

Contains the core business model. Has zero external dependencies.

- **Entities:** Store, ProductListing, Order, Review, AIRecognitionResult, Address, StoreVerification, RefreshToken, Category, Notification, SupportTicket etc.
- **Enums:** AppRole, UserStatus, VerificationStatus, BusinessCategory, ListingStatus, OrderStatus, DocumentTypes
- **Base classes:** BaseEntity (audit fields), ISoftDelete (soft-delete contract)
- All entities extend BaseEntity which gives them Id, CreatedAt, UpdatedAt, CreatedBy, UpdatedBy.

## Application (FoodLoop.Application)

Defines what the system can do without caring how. Depends only on Domain.

- **Commands and Queries** (MediatR): one file per use case, e.g. LoginCommand, RegisterCommand, GetMyStoreQuery
- **DTOs:** request/response shapes for every endpoint
- **Interfaces:** IUnitOfWork, IStoreRepository, IEmailService, IJwtTokenService, IFileStorageService, ICurrentUserService, ILocalizationService
- **Common models:** Result<T>, PagedResult<T>, FileUploadRequest

## Infrastructure (FoodLoop.Infrastructure)

Implements every Application interface. Knows about databases, Identity, JWT, email, file storage, and localisation.

- **Handlers:** one file per command/query, co-located with the feature they implement
- **EF Core:** ApplicationDbContext, all EF configurations, migrations, UnitOfWork, Repository<T>, bespoke repositories (StoreRepository, AddressRepository, RefreshTokenRepository)

- **Identity:** `ApplicationUser, ApplicationRole, IdentitySeeder` (seeds roles at startup)
- **Services:** `JwtTokenService, LocalFileStorageService, NullEmailService, LocalizationService`
- **DI registration:**  
`InfrastructureServiceRegistration.AddInfrastructure()`

## API (FoodLoop.API)

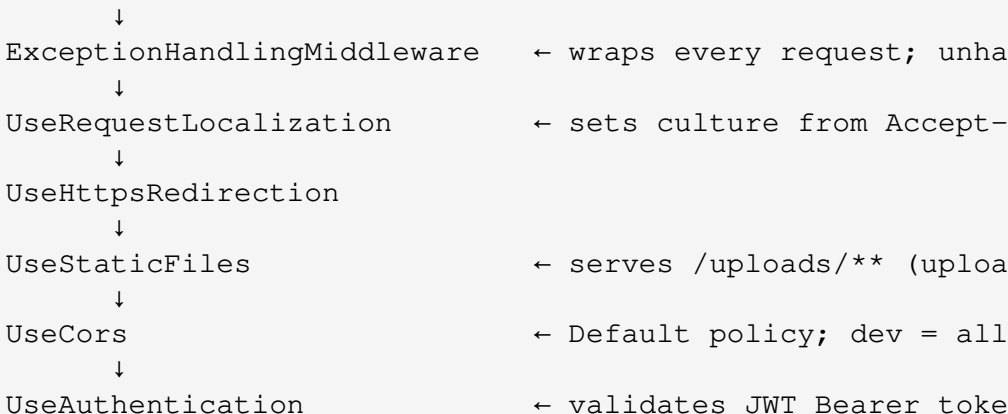
Entry point. Depends on Infrastructure only to wire DI; all business logic stays in Application/Infrastructure.

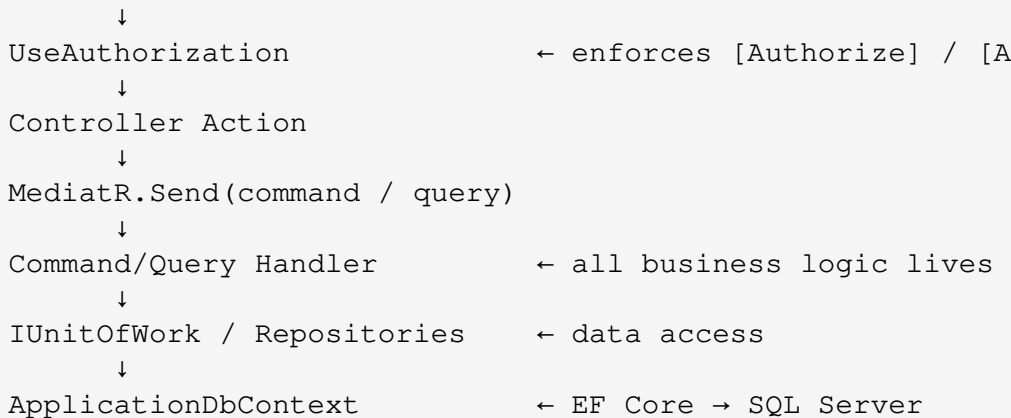
- **Controllers:** `AuthController, UsersController, StoresController, AdminController`
  - **Middleware:** `ExceptionHandlingMiddleware` — converts unhandled exceptions to the standard `{success, message, errors}` envelope
  - **Program.cs:** builds the DI container, configures the middleware pipeline, auto-migrates on development startup, seeds roles
- 

## Request Pipeline

---

Client Request





---

## CQRS with MediatR

---

Every endpoint dispatches a command (mutates state) or query (reads state) through MediatR. Controllers contain no logic — they translate HTTP into a command/query and translate the result back to HTTP.

```
POST /auth/login
→ LoginCommand(request, ip)
→ LoginCommandHandler.Handle()
→ Result<AuthResponse>
→ 200 OK or 401 Unauthorized
```

Handlers live in Infrastructure (not Application) because they depend on `UserManager<ApplicationUser>` and other infrastructure concerns. MediatR is configured to scan both assemblies.

---

## Authentication and Authorisation

---

- **JWT Bearer** — all protected endpoints require a valid access

token in the `Authorization: Bearer <token>` header.

- **RBAC** — roles are seeded at startup and assigned via ASP.NET Core Identity. Endpoints are protected with `[Authorize(Roles = AppRole.Admin)]` etc.
  - **Token expiry**: access token 15 min, refresh token 30 days.
  - Access tokens are stateless (not stored). Refresh tokens are stored in the `RefreshTokens` table and rotated on every use.
- 

## Database

---

- **SQL Server** via Entity Framework Core 10
  - **Migrations**: stored in `FoodLoop.Infrastructure/Migrations/`. In development, `MigrateAsync()` is called at startup automatically. In production, apply migrations via `./scripts/migration-update.sh`.
  - **Soft delete**: deleting any entity that implements `ISoftDelete` sets `IsDeleted = true` and `DeletedAt` rather than issuing a DELETE statement. EF global query filters (where applied) exclude soft-deleted rows automatically.
  - **Audit**: `CreatedAt/UpdatedAt` are stamped by `ApplicationDbContext.SaveChangesAsync` — handlers never set them manually.
  - **Transactions**: multi-step writes (e.g. create user + create store) use `IUnitOfWork.BeginTransactionAsync / CommitTransactionAsync / RollbackTransactionAsync`.
- 

## Localisation (en / ar)

---

- `AddLocalization()` registered in `Program.cs`; `UseRequestLocalization()` is in the middleware pipeline.
- Culture resolved from `Accept-Language` header per request (`en` → English, `ar` → Arabic). Default: `en`.

- All user-facing strings live in `.resx` resource files in `FoodLoop.Infrastructure/Resources/:`
    - `FoodLoop.Infrastructure.Resources.Messages.en.resx`
    - `FoodLoop.Infrastructure.Resources.Messages.ar.resx`
  - `ILocalizationService` is injected into every handler that needs to return user-facing text. No hardcoded English strings exist in handlers or controllers.
- 

## File Storage

---

- `IFileStorageService` abstracts file persistence.
  - Current implementation: `LocalFileStorageService` — saves to `wwwroot/uploads/{folder}/{guid}.ext` and returns a relative URL (`/uploads/...`) served by `UseStaticFiles`.
  - Designed to be swapped for S3 / Azure Blob Storage without changing any Application-layer code — only the DI registration changes.
- 

## API Response Envelope

---

All endpoints return the same envelope shape:

```
{
  "success": true,
  "data": { ... },
  "message": null,
  "errors": []
}
```

```
{
  "success": false,
  "data": null,
```

```
"message": "Email is already registered.",
"errors": ["Email is already registered."]
}
```

`ExceptionHandlerMiddleware` converts unhandled exceptions to this shape:

- `NotFoundException` → 404
- `ForbiddenAccessException` → 403
- `UnauthorizedAccessException` → 401
- `ArgumentException` → 400
- Anything else → 500 (message hidden from client, full stack logged server-side)

---

## CORS

| Environment                      | Policy                                                                        |
|----------------------------------|-------------------------------------------------------------------------------|
| Development                      | All origins, all headers, all methods, credentials allowed                    |
| Production                       | Origins from <code>appsettings.json</code> → <code>Cors:AllowedOrigins</code> |
| Fallback (no origins configured) | <code>AllowAnyOrigin</code> , no credentials (safe fallback)                  |

Currently allowed production origins:

- `http://localhost:3000`
- `http://localhost:5173`
- `https://web-nine-ivory-36.vercel.app`

---

## Key Configuration (appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=...;Database=db60802;...
  },
  "Jwt": {
    "Issuer": "FoodLoop.API",
    "Audience": "FoodLoop.Clients",
    "Secret": "<32+ char secret>",
    "AccessTokenExpirationMinutes": 15,
    "RefreshTokenExpirationDays": 30
  },
  "Cors": {
    "AllowedOrigins": ["http://localhost:3000", "https://"]
  }
}
```

---

## Dependency Injection Overview

---

| Interface            | Implementation                    | Lifetime  |
|----------------------|-----------------------------------|-----------|
| IUnitOfWork          | UnitOfWork                        | Scoped    |
| IJwtTokenService     | JwtTokenService                   | Scoped    |
| ICurrentUserService  | CurrentUserService                | Scoped    |
| IEmailService        | NullEmailService (dev stub)       | Scoped    |
| IFileStorageService  | LocalFileStorageService           | Scoped    |
| ILocalizationService | LocalizationService               | Scoped    |
| IAuthTokenIssuer     | AuthTokenIssuer                   | Scoped    |
| ApplicationDbContext | EF Core SQL Server                | Scoped    |
| All MediatR handlers | Auto-scanned from both assemblies | Transient |

---



# Current Endpoints (Sprint 1)

---

## Auth (/auth)

| Method | Path                      | Auth |
|--------|---------------------------|------|
| POST   | /auth/register            | None |
| POST   | /auth/login               | None |
| POST   | /auth/refresh             | None |
| POST   | /auth/logout              | None |
| POST   | /auth/forgot-password     | None |
| POST   | /auth/reset-password      | None |
| POST   | /auth/resend-verification | None |

## Users (/users)

| Method | Path                     | Auth              |
|--------|--------------------------|-------------------|
| GET    | /users/me                | Any authenticated |
| PATCH  | /users/me                | Any authenticated |
| PATCH  | /users/me/preferences    | Any authenticated |
| GET    | /users/me/addresses      | Any authenticated |
| POST   | /users/me/addresses      | Any authenticated |
| PATCH  | /users/me/addresses/{id} | Any authenticated |

|        |                          |                   |
|--------|--------------------------|-------------------|
| DELETE | /users/me/addresses/{id} | Any authenticated |
| GET    | /users                   | Admin only        |
| GET    | /users/{id}              | Admin only        |
| POST   | /users                   | Admin only        |
| PATCH  | /users/{id}              | Admin only        |
| DELETE | /users/{id}              | Admin only        |

## Stores (/stores)

| Method | Path                 | Auth                                     |
|--------|----------------------|------------------------------------------|
| GET    | /stores/me           | Merchant / Charity                       |
| PATCH  | /stores/me           | Merchant / Charity                       |
| PATCH  | /stores/me/location  | Merchant / Charity                       |
| POST   | /stores/me/documents | <b>None</b> (email identifies the store) |

## Admin (/admin)

| Method | Path                      | Auth                                  |
|--------|---------------------------|---------------------------------------|
| GET    | /admin/stores/pending     | <b>None</b> (open for admin frontend) |
| GET    | /admin/stores/{id}        | <b>None</b> (open for admin frontend) |
| PATCH  | /admin/stores/{id}/verify | Admin only                            |
| PATCH  | /admin/users/{id}/status  | Admin only                            |

GET

/admin/users/{id}/  
activity-log

Admin only